

Lab 3

Minimax, Alpha–Beta Pruning and Max-N

Objective

Implement and analyse adversarial search using:

- Minimax
- Alpha–Beta pruning
- Max-N for multiple players

Problem Statement

Two roommates, A and B , are spending their Sunday together.

- A is a 9-pointer and follows: “*Kal kare so aaj kare.*”
- B is a 6-pointer and follows: “*Never postpone joy.*”
- They must decide how to spend their Sunday.
- Possible activities include coding assignments, exam preparation, research work, sleep, cooking, eating outside, watching a movie, and playing games.
- Assign a reasonable duration to each activity.

Tasks

1. Define the utility model

- Assign utility values to the activities according to the personalities of A and B .
- Clearly state and justify your assumptions.

2. Formulate the game

- Consider A as MAX and B as MIN.
- Define the state representation.
- Define the legal actions.
- Define the terminal states. [Hint: Define a terminal state as a state where no further legal activity can be selected within the remaining Sunday time.]
- Define the utility of a terminal state.

3. Construct the game tree

Use at least three decision levels.

A possible structure is:

$$A \rightarrow B \rightarrow A$$

For example:

Coding $\rightarrow$ Game $\rightarrow$ Reading

may be one complete path through the game tree.

4. Implement Minimax

- Evaluate the game tree.
- Determine the optimal decision for A .
- Display the selected path and final utility.

5. Implement Alpha-Beta pruning

- Maintain α and β values.
- Use the pruning condition:

$$\alpha \geq \beta.$$

- Display every branch that is pruned.

6. Compare Minimax and Alpha-Beta

Record:

Algorithm	Nodes Generated	Nodes Evaluated	Nodes Pruned
Minimax	?	?	0
Alpha-Beta	?	?	?

Verify that both algorithms produce the same final decision.

7. Investigate move ordering [optional]

Run Alpha-Beta with:

- original child ordering,
- good child ordering,
- poor child ordering.

Compare the number of nodes evaluated and pruned.

8. Multiplayer extension

Introduce a third roommate C .

Use the turn order:

$$A \rightarrow B \rightarrow C \rightarrow A.$$

Define the terminal utility as:

$$U(s) = (u_A, u_B, u_C).$$

Implement the Max-N algorithm.

At a node belonging to player i , select the child that maximizes the i -th component of the utility vector.

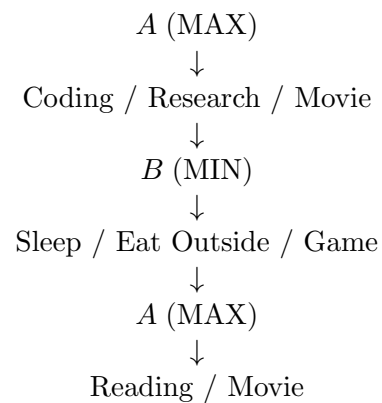
9. Generalization

Extend the program to N roommates:

$$U(s) = (u_1, u_2, \dots, u_N).$$

Example Game Structure

A simple three-level game may be:



This gives:

$$3 \times 3 \times 2 = 18$$

terminal states.

Students must assign their own terminal utilities.

Important Observation

The real roommate problem may not be strictly zero-sum.

Discuss whether:

$$U = A - B$$

is a reasonable representation of the situation.

Consider whether the roommates could instead cooperate or compromise.